

## PROJECT: Scalable Web Application with Load Balancing

**Services:** Amazon EC2, Application Load Balancer (ELB), Auto Scaling Group, Launch Template, Target Group, Amazon CloudWatch, Amazon VPC, Security Groups, and IAM.

**Project Goal:** To build a web application on AWS that automatically handles traffic by sharing load across servers and increasing or decreasing servers as needed.

**Skills:** EC2 management, load balancing, auto scaling, CloudWatch monitoring, AWS networking, and basic cloud architecture.

**Definition:** This project focuses on designing and deploying a scalable and highly available web application using AWS cloud services. It uses load balancing to distribute incoming user traffic evenly across multiple servers and auto scaling to automatically increase or decrease the number of servers based on system load. The project demonstrates how cloud infrastructure can provide reliability, performance, and cost efficiency while handling changing traffic demands.

**Scope of the Project:** This project focuses on creating a **scalable and highly available web application infrastructure using AWS cloud services**. It demonstrates how cloud resources can automatically handle changes in user traffic while maintaining performance and reliability. The project mainly covers infrastructure setup, monitoring, and scalability using managed AWS services.

### Methodologies:

#### 1. Single EC2 Instance Method

In this method, the web application is hosted on a single EC2 instance.

- **Limitation:** No scalability and high risk of downtime
- **Why not preferred:** If the server fails, the application stops

#### 2. Multiple EC2 Instances Without Auto Scaling

#### 3. Fixed Number of EC2 Instances

A fixed number of EC2 servers always run regardless of traffic.

- **Limitation:** Wastes cost during low traffic
- **Why not preferred:** Not flexible or cost-efficient

### Chosen Method: Scalable Web Application with Load Balancing

EC2 + Load Balancer + Auto Scaling

This method is better because:

- It **automatically scales** based on traffic
- Provides **high availability** and fault tolerance
- **Cost-efficient**, as resources run only when needed
- **Easy to understand and implement**
- Widely used in **real-world production environments**

**Services Used in this Project:****1. Amazon EC2 (Elastic Compute Cloud)**

**Definition:** EC2 provides virtual servers in the cloud to run applications.

**Purpose:** To host applications without using physical hardware.

**Role in the Project:** EC2 instances run the web application and handle user requests.

**2. Application Load Balancer (ALB)**

**Definition:** ALB distributes incoming traffic across multiple servers.

**Purpose:** To improve availability and performance.

**Role in the Project:** ALB receives user requests and forwards them to healthy EC2 instances.

**3. Target Group**

**Definition:** A target group is a group of resources connected to a load balancer.

**Purpose:** To monitor health and route traffic correctly.

**Role in the Project:** EC2 instances are registered here so ALB can check their health.

**4. Auto Scaling Group (ASG)**

**Definition:** Auto Scaling Group automatically adjusts the number of EC2 instances.

**Purpose:** To handle changes in traffic automatically.

**Role in the Project:** ASG adds EC2 instances during high load and removes them during low load.

**5. Launch Template**

**Definition:** A launch template stores EC2 configuration details.

**Purpose:** To ensure consistent server creation.

**Role in the Project:** Used by Auto Scaling to launch identical EC2 instances.

**6. Amazon CloudWatch**

**Definition:** CloudWatch is a monitoring service for AWS resources.

**Purpose:** To track performance and usage metrics.

**Role in the Project:** Monitors CPU usage and triggers Auto Scaling actions.

**7. Amazon VPC (Virtual Private Cloud)**

**Definition:** VPC provides a private network within AWS.

**Purpose:** To securely host cloud resources.

**Role in the Project:** All EC2 instances and load balancers run inside the VPC.

**8. Security Groups**

**Definition:** Security groups act as virtual firewalls.

**Purpose:** To control inbound and outbound traffic.

**Role in the Project:** Allows HTTP traffic and blocks unauthorized access.

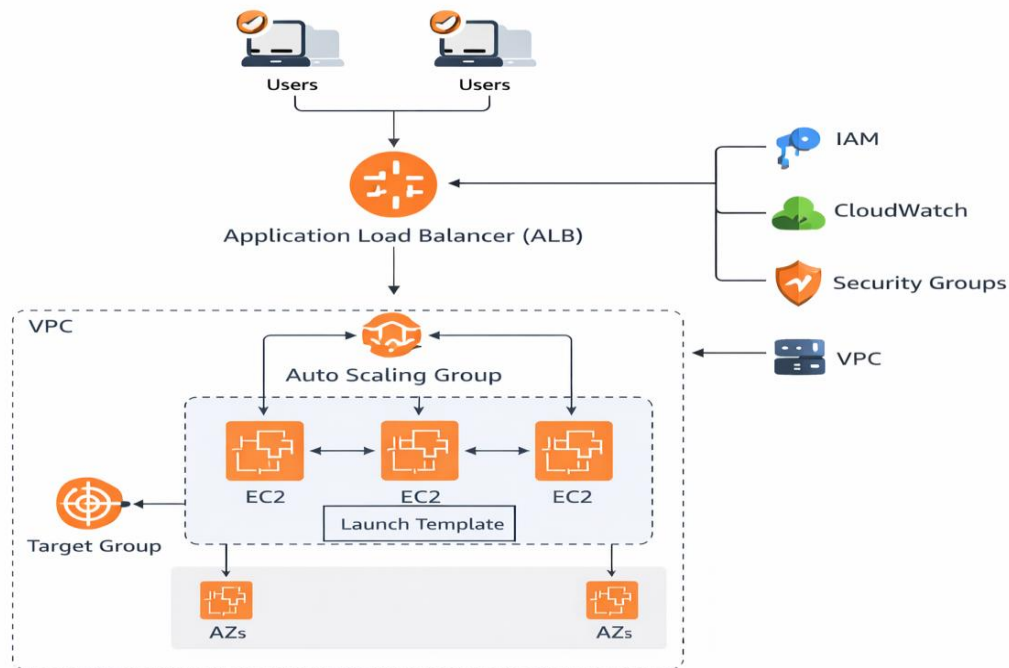
**9. IAM (Identity and Access Management)**

**Definition:** IAM manages permissions for AWS services.

**Purpose:** To ensure secure access and communication.

**Role in the Project:** Allows EC2, Auto Scaling, and CloudWatch to work securely.

## Architecture Diagram



## Implementation and Execution of a Scalable Web Application with Load Balancing

### Phase 1: Create VPC

1. Go to **AWS Console** → **VPC**
2. Click **Create VPC**
3. Select **VPC only**
  - Name: Scalable-Web-VPC
  - IPv4 CIDR block: 10.0.0.0/16
  - IPv6: None Tenancy: Default
4. Click **Create VPC**

The screenshot shows the AWS VPC console interface. At the top, a green banner indicates that the VPC 'vpc-0d89618c353d7f13d / Scalable-Web-VPC' was successfully created. The left sidebar shows the 'VPC dashboard' with a filter set to 'Your VPCs'. The main content area displays the details for the selected VPC:

| Details                                 |                       | Info                                              |                        |
|-----------------------------------------|-----------------------|---------------------------------------------------|------------------------|
| <b>VPC ID</b>                           | vpc-0d89618c353d7f13d | <b>State</b>                                      | Available              |
| <b>DNS resolution</b>                   | Enabled               | <b>Tenancy</b>                                    | default                |
| <b>Main network ACL</b>                 | acl-069e0b118aad6ea12 | <b>Default VPC</b>                                | No                     |
| <b>IPv6 CIDR (Network border group)</b> | -                     | <b>Network Address Usage metrics</b>              | Disabled               |
| <b>Encryption control ID</b>            | -                     | <b>Encryption control mode</b>                    | -                      |
| <b>Block Public Access</b>              | Off                   | <b>DHCP option set</b>                            | dopt-06208c8207636ba54 |
| <b>IPv4 CIDR</b>                        | 10.0.0.0/16           | <b>Route 53 Resolver DNS Firewall rule groups</b> | -                      |
| <b>DNS hostnames</b>                    | Disabled              | <b>Main route table</b>                           | rtb-041071bceb20efee   |
| <b>IPv6 pool</b>                        | -                     | <b>Owner ID</b>                                   | 207454478223           |

## Phase 2: Create Subnets (Public + Private)

1. Go to **VPC** → **Subnets** → **Create subnet**

2. Create Public Subnet 1

- VPC: Scalable-Web-VPC
- Name: Public-Subnet-1
- AZ: ap-south-1a
- CIDR: 10.0.1.0/24

3. Create Public Subnet 2

- Name: Public-Subnet-2
- AZ: ap-south-1b
- CIDR: 10.0.2.0/24

4. Create Private Subnet 1

- Name: Private-Subnet-1
- AZ: ap-south-1a
- CIDR: 10.0.3.0/24

5. Create Private Subnet 2

- Name: Private-Subnet-2
- AZ: ap-south-1b
- CIDR: 10.0.4.0/24

**Subnets (4)** Info

Find subnets by attribute or tag

Subnet ID : subnet-0fca43674a45a84b1 X Subnet ID : subnet-0186390bbe5e1a0ca X Subnet ID : subnet-03f1d40bd7ce0d1fb X Show more (+1) Clear filters

| <input type="checkbox"/> | Name             | Subnet ID                | State     | VPC                             | Block Public... | IPv4 CIDR   |
|--------------------------|------------------|--------------------------|-----------|---------------------------------|-----------------|-------------|
| <input type="checkbox"/> | Private-Subnet-2 | subnet-0236b0e51bc911552 | Available | vpc-0d89618c353d7f13d   Scal... | Off             | 10.0.4.0/24 |
| <input type="checkbox"/> | Public-Subnet-1  | subnet-0fca43674a45a84b1 | Available | vpc-0d89618c353d7f13d   Scal... | Off             | 10.0.1.0/24 |
| <input type="checkbox"/> | Private-Subnet-1 | subnet-03f1d40bd7ce0d1fb | Available | vpc-0d89618c353d7f13d   Scal... | Off             | 10.0.3.0/24 |
| <input type="checkbox"/> | Public-Subnet-2  | subnet-0186390bbe5e1a0ca | Available | vpc-0d89618c353d7f13d   Scal... | Off             | 10.0.2.0/24 |

## Phase 3: Create Internet Gateway

1. Go to **VPC** → **Internet Gateways**

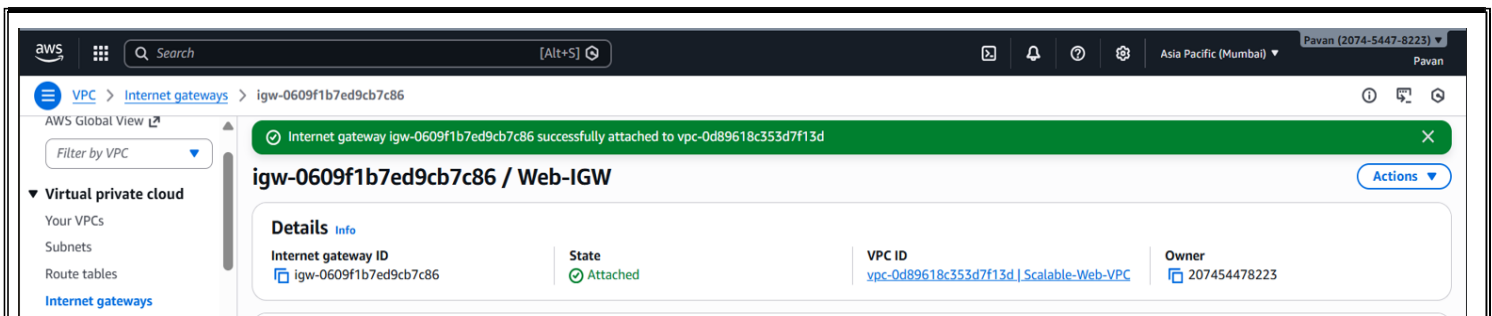
2. Click **Create Internet Gateway**

3. Name: Web-IGW

4. Click **Create**

5. Select IGW → **Actions** → **Attach to VPC**

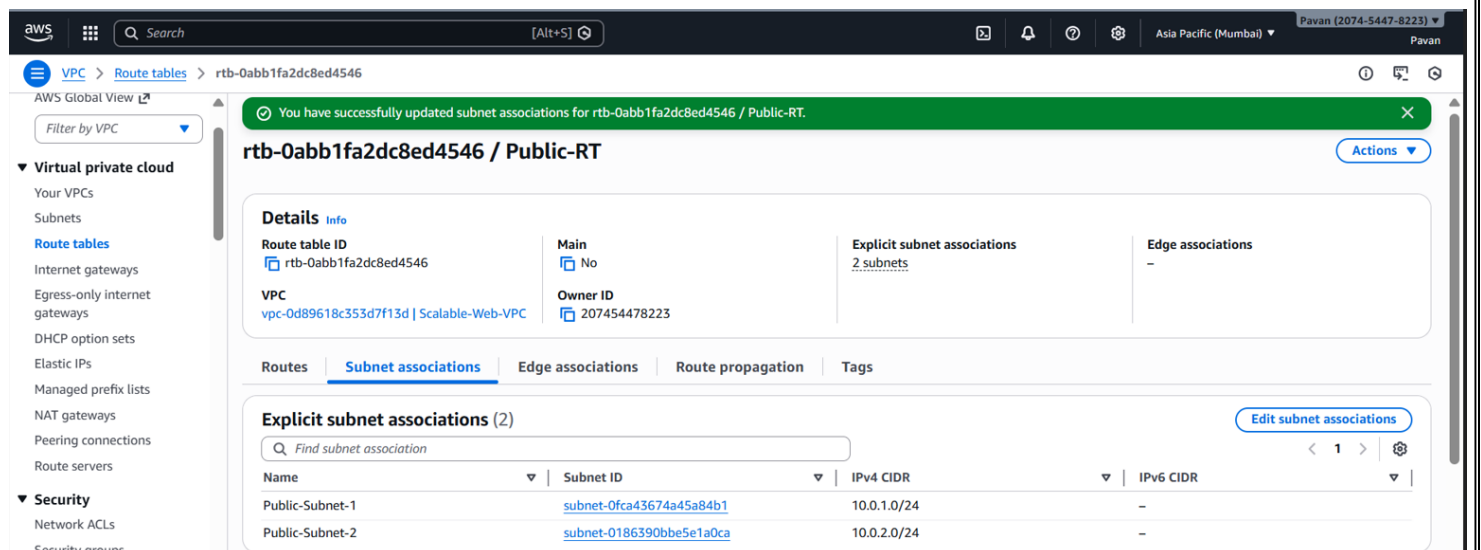
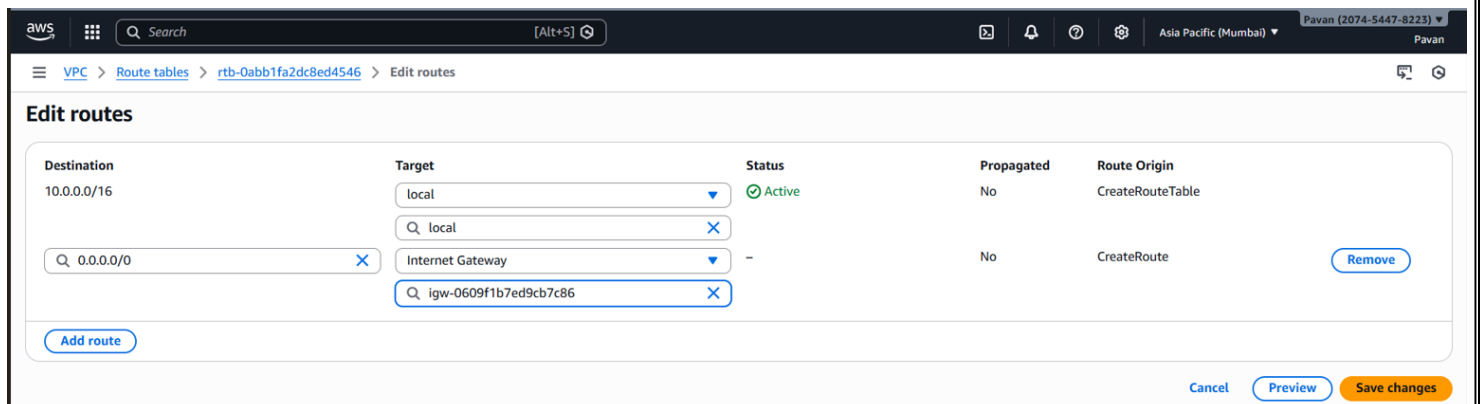
6. Attach to Scalable-Web-VPC



## Phase 4: Configure Route Tables

### Public Route Table

1. Go to **Route Tables**
2. Click **Create route table**
3. Name: `Public-RT`
4. VPC: `Scalable-Web-VPC`
5. Create
6. Go to **Routes** → **Edit routes**
7. Add route:
  - Destination: `0.0.0.0/0`
  - Target: Internet Gateway
8. Save routes
9. **Subnet associations** → **Edit**
10. Associate **Public-Subnet-1** & **Public-Subnet-2**



## Phase 5: Create Security Groups

### 1. Load Balancer Security Group

- Name: **ALB-SG**
- Inbound:
  - HTTP (80) → 0.0.0.0/0
- Outbound: All traffic

The screenshot shows the AWS Management Console interface for the 'sg-0ed61e52c7b0d429f - ALB-SG' security group. A green notification banner at the top states: 'Security group (sg-0ed61e52c7b0d429f | ALB-SG) was created successfully'. The left sidebar shows the navigation menu with 'Security groups' selected under the 'Security' section.

**Details**

|                                      |                                                  |                                                   |                                        |
|--------------------------------------|--------------------------------------------------|---------------------------------------------------|----------------------------------------|
| <b>Security group name</b><br>ALB-SG | <b>Security group ID</b><br>sg-0ed61e52c7b0d429f | <b>Description</b><br>load balancer SG            | <b>VPC ID</b><br>vpc-0d89618c353d7f13d |
| <b>Owner</b><br>207454478223         | <b>Inbound rules count</b><br>1 Permission entry | <b>Outbound rules count</b><br>1 Permission entry |                                        |

**Inbound rules (1)**

| Name | Security group rule ID | IP version | Type | Protocol | Port range | Source    |
|------|------------------------|------------|------|----------|------------|-----------|
| -    | sgr-02acbfc6189cf3269  | IPv4       | HTTP | TCP      | 80         | 0.0.0.0/0 |

**Outbound rules (1)**

| Name | Security group rule ID | IP version | Type        | Protocol | Port range | Destination |
|------|------------------------|------------|-------------|----------|------------|-------------|
| -    | sgr-02f5b93abc5a24d96  | IPv4       | All traffic | All      | All        | 0.0.0.0/0   |

### 2. EC2 Security Group

- Name: **Web-EC2-SG**
- Inbound:
  - HTTP (80) → **ALB-SG**
  - SSH (22) → **Your IP**
- Outbound: All traffic

The screenshot shows the AWS Management Console interface for the 'sg-0270b20fd1798e35b - Web-EC2-SG' security group. A green notification banner at the top states: 'Security group (sg-0270b20fd1798e35b | Web-EC2-SG) was created successfully'. The left sidebar shows the navigation menu with 'Security groups' selected under the 'Security' section.

**Details**

|                                          |                                                    |                                                   |                                        |
|------------------------------------------|----------------------------------------------------|---------------------------------------------------|----------------------------------------|
| <b>Security group name</b><br>Web-EC2-SG | <b>Security group ID</b><br>sg-0270b20fd1798e35b   | <b>Description</b><br>EC-2 SG                     | <b>VPC ID</b><br>vpc-0d89618c353d7f13d |
| <b>Owner</b><br>207454478223             | <b>Inbound rules count</b><br>2 Permission entries | <b>Outbound rules count</b><br>1 Permission entry |                                        |

**Inbound rules (2/2)**

| Name | Security group rule ID | IP version | Type | Protocol | Port range | Source               |
|------|------------------------|------------|------|----------|------------|----------------------|
| -    | sgr-00f4006bcc6ce28d3  | -          | HTTP | TCP      | 80         | sg-0ed61e52c7b0d429f |
| -    | sgr-084d4f3fa23f384cf  | IPv4       | SSH  | TCP      | 22         | 122.168.1.1          |

## Outbound rules (1/1)

[Manage tags](#) [Edit outbound rules](#)

| <input checked="" type="checkbox"/> | Name | Security group rule ID | IP version | Type        | Protocol | Port range | Destination |
|-------------------------------------|------|------------------------|------------|-------------|----------|------------|-------------|
| <input checked="" type="checkbox"/> | -    | sgr-081d5d12c21e9f886  | IPv4       | All traffic | All      | All        | 0.0.0.0/0   |

## 3. RDS Security Group

- Name: **RDS-SG**
- Inbound:
  - MySQL (3306) → **Web-EC2-SG**
- Outbound: All traffic

aws [Search](#) [Alt+S] Asia Pacific (Mumbai) Pavan (2074-5447-8223)

VPC > Security Groups > sg-0142157dba3f05a81 - RDS-SG

Filter by VPC

**Virtual private cloud**

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

**Security**

- Network ACLs
- Security groups**

**Security group (sg-0142157dba3f05a81 | RDS-SG) was created successfully**

**sg-0142157dba3f05a81 - RDS-SG** [Actions](#)

**Details**

|                                      |                                                  |                                                   |                                        |
|--------------------------------------|--------------------------------------------------|---------------------------------------------------|----------------------------------------|
| <b>Security group name</b><br>RDS-SG | <b>Security group ID</b><br>sg-0142157dba3f05a81 | <b>Description</b><br>RDS-SG                      | <b>VPC ID</b><br>vpc-0d89618c353d7f13d |
| <b>Owner</b><br>207454478223         | <b>Inbound rules count</b><br>1 Permission entry | <b>Outbound rules count</b><br>1 Permission entry |                                        |

[Inbound rules](#) [Outbound rules](#) [Sharing](#) [VPC associations](#) [Related resources - new](#) [Tags](#)

**Inbound rules (1/1)** [Manage tags](#) [Edit inbound rules](#)

| <input checked="" type="checkbox"/> | Name | Security group rule ID | IP version | Type         | Protocol | Port range | Source  |
|-------------------------------------|------|------------------------|------------|--------------|----------|------------|---------|
| <input checked="" type="checkbox"/> | -    | sgr-0d27df8269e89c41f  | -          | MySQL/Aurora | TCP      | 3306       | sg-027c |

**Outbound rules (1/1)** [Manage tags](#) [Edit outbound rules](#)

| <input checked="" type="checkbox"/> | Name | Security group rule ID | IP version | Type        | Protocol | Port range | Destination |
|-------------------------------------|------|------------------------|------------|-------------|----------|------------|-------------|
| <input checked="" type="checkbox"/> | -    | sgr-081d5d12c21e9f886  | IPv4       | All traffic | All      | All        | 0.0.0.0/0   |

## Phase 6: Create RDS (MySQL)

1. Go to **RDS** → **Create database**
2. Choose **Standard create**
3. Engine: **MySQL**
4. Version: default
  - DB identifier: webapp-db
  - Username: admin
  - Password: set password
5. EC2 → Create Instance
  - Class: db.t3.micro
  - Storage: 20 GB

## Connectivity

- VPC: Scalable-Web-VPC
- Subnet group: Private subnets
- Public access: ✗
- Security group: RDS-SG

## 6. Create database

Wait 5–10 minutes

The screenshot shows the AWS Management Console for the 'webapp-db' database instance. A green banner at the top states 'Successfully created database webapp-db'. The instance summary shows it is 'Available' with a class of 'db.t3.micro' and 0 connections. The 'Connectivity & security' tab is active, showing connection methods like 'Code snippets' and 'CloudShell'. The 'Internet access gateway' is disabled, and 'IAM Authentication' is also disabled. A 'Code copied' notification is visible on the right.

The screenshot shows the AWS Management Console for the 'webapp-db-subnet-group' subnet group. A green banner at the top states 'Successfully created webapp-db-subnet-group'. The subnet group details show it is 'Complete' and associated with the VPC 'vpc-0d89618c353d7f13d'. The 'Create DB subnet group' button is highlighted in orange.

## Phase 7: Create S3 Bucket for Static Files

1. Go to S3 → Create bucket
2. Bucket name: **webapp-static-pavan**
3. Region: same region
4. Uncheck **Block all public access**
5. Create bucket
6. Enable Static Website Hosting
  - Properties → Static website hosting → Enable
  - Index document: `index.html`
7. Upload Files
  - **style.css**
  - **image.png**



**Successfully created bucket "webapp-static-pavan"**  
To upload files and folders, or to configure additional bucket settings, choose [View details](#).

**General purpose buckets** (1) [Info](#)

Buckets are containers for data stored in S3.

| Name                                | AWS Region                       | Creation date                          |
|-------------------------------------|----------------------------------|----------------------------------------|
| <a href="#">webapp-static-pavan</a> | Asia Pacific (Mumbai) ap-south-1 | February 4, 2026, 11:42:18 (UTC+05:30) |

**Static website hosting**  
Use this bucket to host a website or redirect requests. [Learn more](#)

**S3 static website hosting**  
Enabled

**Hosting type**  
Bucket hosting

**Bucket website endpoint**  
When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)  
<http://webapp-static-pavan.s3-website-ap-south-1.amazonaws.com>

**Upload succeeded**  
For more information, see the [Files and folders](#) table.

**Upload: status**

After you navigate away from this page, the following information is no longer available.

**Summary**

| Destination                              | Succeeded                 | Failed            |
|------------------------------------------|---------------------------|-------------------|
| <a href="#">s3://webapp-static-pavan</a> | 2 files, 2.1 MB (100.00%) | 0 files, 0 B (0%) |

**Files and folders** (2 total, 2.1 MB)

| Name                                   | Folder | Type       | Size    | Status    | Error |
|----------------------------------------|--------|------------|---------|-----------|-------|
| <a href="#">134114507095561596.jpg</a> | -      | image/jpeg | 2.1 MB  | Succeeded | -     |
| <a href="#">style.css</a>              | -      | text/css   | 359.0 B | Succeeded | -     |

## Phase 8: Create IAM Role for EC2

1. Go to **IAM** → **Roles** → **Create role**
2. Service: **EC2**
3. Policies:
  - AmazonS3ReadOnlyAccess
  - CloudWatchAgentServerPolicy
4. Role name: **EC2-Web-Role**

**Role EC2-Web-Role created.**

**EC2-Web-Role** [Info](#)

Allows EC2 instances to call AWS services on your behalf.

**Summary**

| Creation date                        | ARN                                                         | Instance profile ARN                                                    |
|--------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------|
| February 04, 2026, 11:53 (UTC+05:30) | <a href="#">arn:aws:iam::207454478223:role/EC2-Web-Role</a> | <a href="#">arn:aws:iam::207454478223:instance-profile/EC2-Web-Role</a> |

**Permissions policies (2)** [Info](#)

You can attach up to 10 managed policies.

| Policy name                                 | Type        | Attached entities |
|---------------------------------------------|-------------|-------------------|
| <a href="#">AmazonS3ReadOnlyAccess</a>      | AWS managed | 1                 |
| <a href="#">CloudWatchAgentServerPolicy</a> | AWS managed | 1                 |

## Phase 9: Create Launch Template

### 1. Go to EC2 → Launch Templates

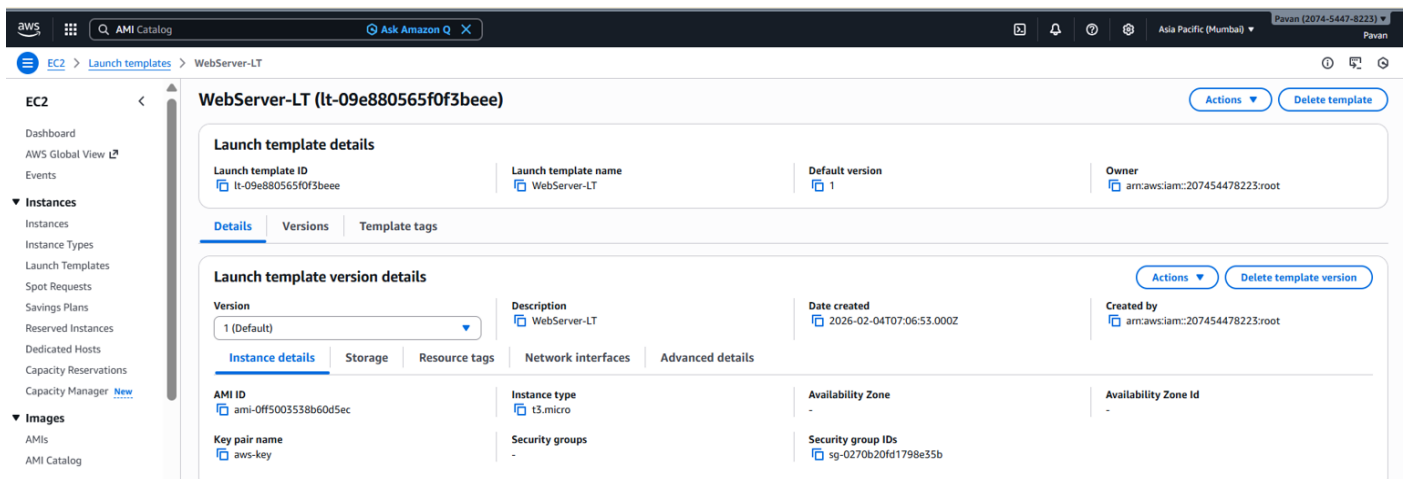
### 2. Create launch template

- Name: **WebApp-LT**
- AMI: Amazon Linux 2
- Instance type: **t3.micro**
- Key pair: your key
- Security group: **Web-EC2-SG**
- IAM role: **EC2-Web-Role**

### 3. Additional Settings → User Data Script → Paste this code→

```
#!/bin/bash
yum update -y
yum install -y httpd php mysql
systemctl start httpd
systemctl enable httpd

cat <<EOF > /var/www/html/index.php
<?php
echo "<h1>Scalable Web Application</h1>";
echo "<p>Instance ID: " . gethostname() . "</p>";
?>
EOF
```



## Phase 10: Create Application Load Balancer

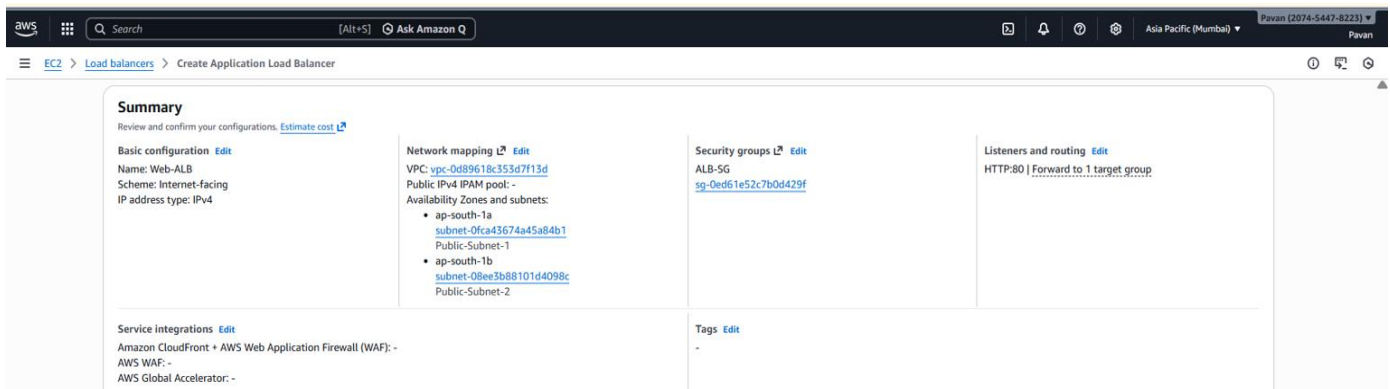
### 1. Go to EC2 → Load Balancers

### 2. Create Load Balancer → Application LB

- Name: **Web-ALB**
- Scheme: Internet-facing
- IP type: IPv4
- Subnets: Public-Subnet-1 & 2
- Security group: **ALB-SG**

### 3. Target Group

- Type: Instance
- Name: **Web-TG**
- Health check path: /



## Phase 11: Create Auto Scaling Group

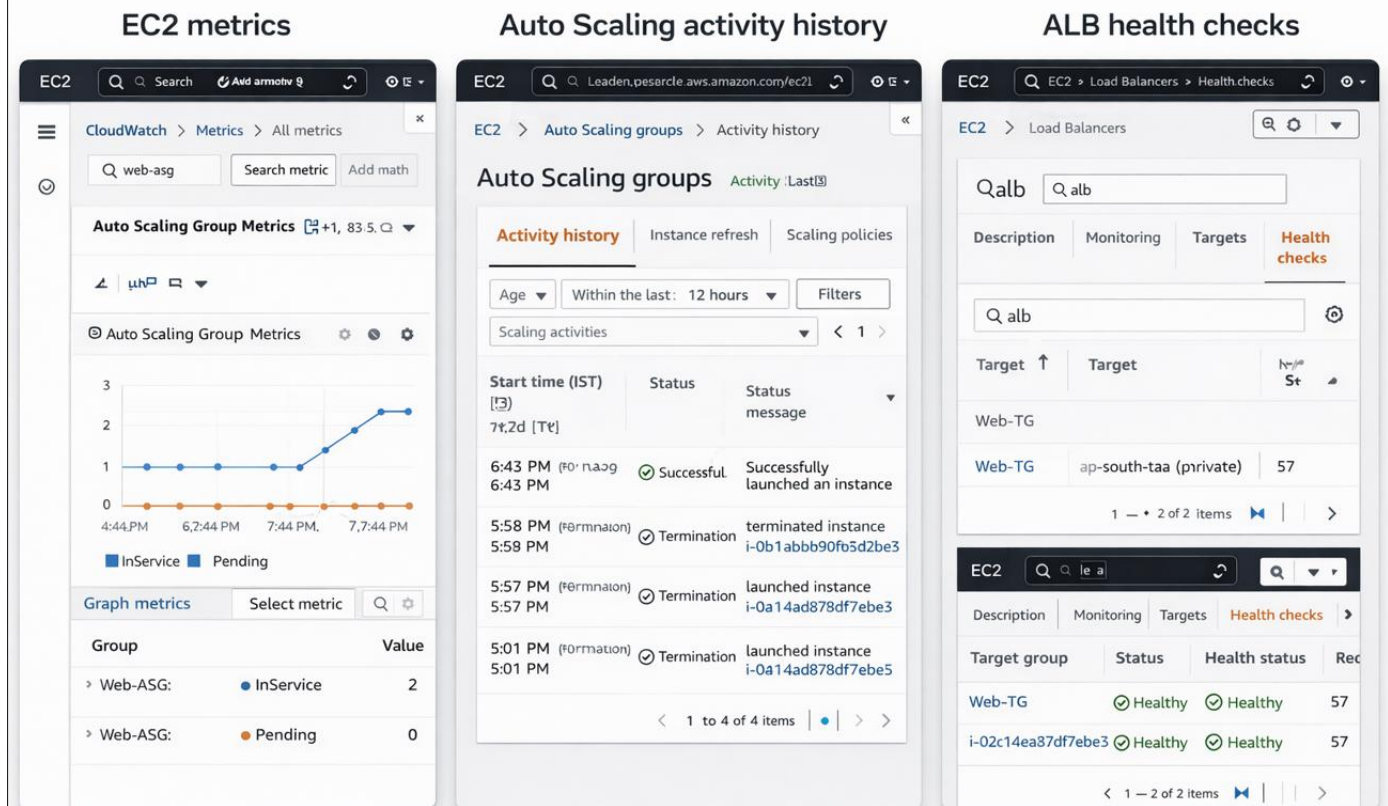
1. Go to **Auto Scaling Groups**
2. Create ASG
3. Launch template: WebApp-LT
4. Configuration
  - Min: 1
  - Desired: 2
  - Max: 4
  - Subnets: Public subnets
  - Attach to target group: Web-TG
5. Scaling Policy
  - CPU > 60% → Scale out
  - CPU < 30% → Scale in

The screenshot shows the AWS Management Console for the 'Web-ASG' Auto Scaling Group. The left sidebar contains navigation links for Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area displays the 'Web-ASG Capacity overview' with a table showing 'Desired capacity' (2), 'Scaling limits' (1 - 4), 'Desired capacity type' (Units (number of instances)), and 'Status' (-). Below this, the 'Launch template' section shows details for 'WebServer-LT', including AMI ID, Instance type (t3.micro), Security groups, and Key pair name (aws-key).

The screenshot shows the AWS Management Console for the 'Web-ASG' Auto Scaling Group configuration page. The left sidebar contains navigation links for Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area displays the 'Network' section with 'Availability Zones' (aps1-az3 (ap-south-1b), aps1-az1 (ap-south-1a)), 'Subnet ID' (subnet-04cad0b6539ade494, subnet-03f1d40bd7ce0d1fb), and 'Availability Zone distribution' (Balanced best effort). Below this, the 'Instance type requirements' section shows 'Your Auto Scaling group adheres to the launch template for purchase option and instance type.' and the 'Health checks' section shows 'Health check type' (EC2) and 'Health check grace period' (300).

## Phase 12 : Monitoring

- CloudWatch → EC2 metrics
- Auto Scaling activity history
- ALB health checks



## Troubleshooting:

### Issue 1: Load Balancer Creation Failed

#### Issue Occurred:

Application Load Balancer was not getting created.

#### Cause:

The AWS account had restrictions or limits that did not allow creating a load balancer.

#### Solution:

Verified AWS account status and region settings. Ensured correct VPC and subnet selection. After correction, load balancer creation worked successfully.

### Issue 2: EC2 Instances Not Registering with Target Group

#### Issue Occurred:

EC2 instances were not appearing in the target group.

#### Cause:

Incorrect target type or wrong VPC selected during target group creation.

#### Solution:

Recreated the target group with correct VPC and instance target type, then registered instances again.

### Issue 3: Auto Scaling Not Triggering

**Issue Occurred:**

Auto Scaling Group was not launching new EC2 instances.

**Cause:**

CloudWatch alarm threshold was set too high, and CPU usage never crossed the limit.

**Solution:**

Adjusted CloudWatch alarm threshold to realistic CPU values (e.g., 60%) so scaling actions could trigger.

**Conclusion**

In this project, a scalable and highly available web application was successfully implemented using AWS cloud services. Load balancing ensured that user traffic was evenly distributed across multiple servers, improving performance and reliability. Auto Scaling automatically adjusted the number of servers based on demand, which helped reduce costs and avoid manual intervention. Monitoring through CloudWatch enabled better visibility into system performance. Overall, this project demonstrates how AWS can be effectively used to build reliable, flexible, and cost-efficient cloud-based applications suitable for real-world use...